Lab 2: Memory Hierarchy
Progress Summary

October 21, 2026

1 Lab premise

Lab 2 asks us to build a software model of a processor memory hierarchy and use it to study cache design. The long-term goal is to choose a practical two-level cache organization that provides low Average Memory Access Time (AMAT) while keeping area and power reasonable.

The lab progresses through four stages:

1. Implement a configurable set-associative cache.
2. Compare random, LRU, bit-PLRU, and other replacement policies.
3. Build a parameterized L1/L2 cache hierarchy.
4. Search the design space under a total 5 MB cache-storage budget, using benchmark results and Destiny hardware-cost estimates.

The central tradeoff is that larger blocks, greater associativity, and larger caches can reduce misses, but can also increase access latency, area, leakage power, dynamic power, and tag-storage overhead.

2 Repository and execution model

The project contains a Pin-based instrumentation framework:

- `memory-hierarchy.cpp` instruments every executed memory operand. Reads call `mhLoad`; writes call `mhStore`.
- `cache.h` declares the cache model and the interface called by the Pin instrumentation.
- `cache.cpp` implements the cache model.
- `makefile` and `makefile.rules` describe how the Pin tool is built.
- The PDF writeup describes the required experiments and analysis.

The runtime flow is:

```
benchmark instruction
        -> Pin memory instrumentation
        -> mhLoad() or mhStore()
        -> simulated cache
        -> statistics
        -> memhier.stats at program termination
```

3 Work completed

3.1 `cache.h`

The header now defines a first single-cache abstraction:

- `CacheLine`, containing a valid bit and tag;
- `CacheStats`, containing accesses, hits, and misses;
- `Cache`, parameterized by capacity, block size, and associativity;
- vectors representing sets and ways;
- a per-set victim pointer for temporary round-robin replacement;
- `mhInit()` and `mhPrintStats()` for Pin integration;
- the existing `mhLoad()` and `mhStore()` entry points.

3.2 `cache.cpp`

The first working cache implementation is complete. It performs the following:

1. Validates cache dimensions.
2. Computes the number of blocks and sets.
3. Allocates each set with the requested number of ways.
4. Converts an address to a block address.
5. Computes the set index and tag:

```
block address = address / block size
set index     = block address % number of sets
tag           = block address / number of sets
```

6. Searches every way in the selected set for a matching valid tag.
7. Records hits and misses.
8. Uses an invalid way before evicting a valid line.
9. Uses simple round-robin replacement when a set is full.
10. Splits an access that crosses a block boundary into multiple block accesses.
11. Prints accesses, hits, misses, and miss rate.

For example, an 8-byte access beginning at byte 60 in a cache with 16-byte blocks touches two simulated cache blocks.

3.3 `memory-hierarchy.cpp`

The existing Pin instrumentation was preserved. The following integration was added:

- After Pin parses command-line arguments, `mhInit()` is called.
- `-l1c` is converted from kilobytes to bytes.
- `-l1b` supplies the block size.
- `-l1a` supplies the associativity.
- At program termination, `Fini()` calls `mhPrintStats()`.

The current default configuration is equivalent to:

```
capacity:      2 KB
block size:    64 bytes
associativity: 2-way
```

4 Validation performed

The implementation was compiled independently of Pin with:

```
c++ -std=c++11 -Wall -Wextra -pedantic -I. cache.cpp cache_test.cpp
```

A deterministic test exercised:

- a repeated access to the same block;
- an access within an already cached block;
- a store crossing a block boundary;
- the `mhInit()` and `mhPrintStats()` interface.

The resulting statistics were:

```
accesses: 4
hits: 1
misses: 3
miss_rate: 0.75
```

This confirms that the cache can be configured through the public interface, that repeated block accesses hit, and that a boundary-crossing access is counted once for each block touched.

5 Current status and limitations

The project currently has a correct first single-cache milestone, but it is not yet the completed lab. The following work remains:

- Run the actual Pin tool on a small program and verify `memhier.stats`.
- Replace the temporary round-robin policy with random, LRU, and bit-PLRU.

- Add load/store policy decisions such as write-back or write-through and write-allocate behavior.
- Add L2 and DRAM behavior.
- Add latency and AMAT accounting.
- Connect Destiny area, leakage-power, dynamic-power, and latency data.
- Build benchmark automation, design-space search, Pareto-frontier plots, and sensitivity studies.

The actual Pin build/run has not yet been performed in this work session because the Pin kit location and executable test workload have not been configured. The cache implementation has instead been validated with a standalone C++ test.

6 Recommended next step

The next task is to verify the instrumented Pin path with a very small test program. After that, use the existing L1 knobs for experiments. Only once the single-cache hit/miss behavior is trusted should replacement policies, L2, AMAT, and design-space exploration be added.